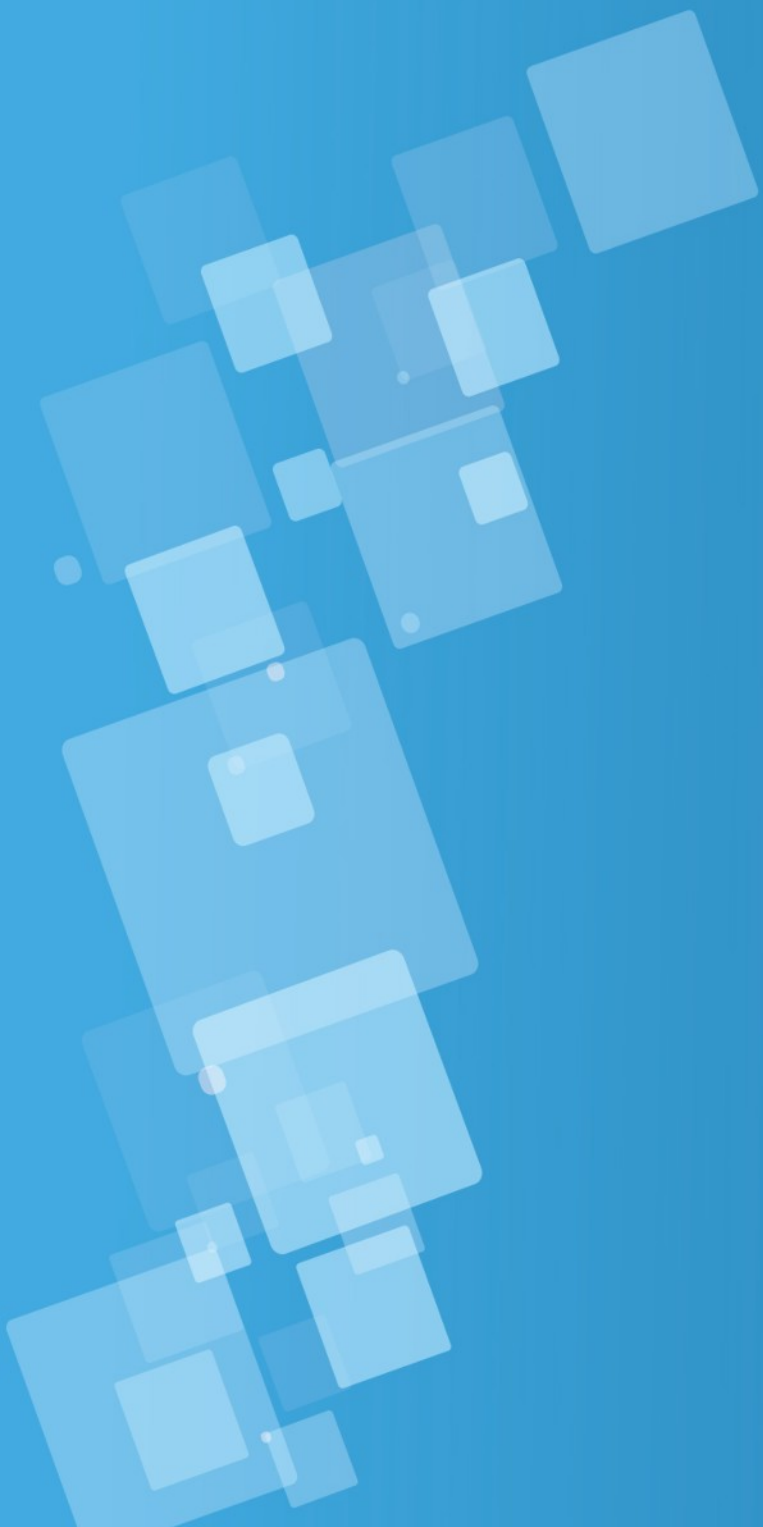




Técnicas Digitales II

Lenguaje de Descripción de Hardware

Dr.Ing.Steiner Guillermo

- Bibliografía

Harris & Harris. Digital design and computer architecture: ARM edition. Elsevier, 2015. Capítulo 4



Lenguaje de descripción de Hardware

- Hasta ahora, nos hemos centrado en el diseño de circuitos digitales combinacionales y secuenciales a nivel esquemático.
- Encontrar el conjunto óptimo de compuertas lógicas para realizar una determinada función es complicado y propenso a errores.
- Una herramienta de diseño asistido por computadora (CAD) puede hacer esto mucho mas sencillo.
- Las especificaciones para este diseño se realiza en un Lenguaje de Descripción de Hardware (HDL)



Lenguaje de descripción de Hardware

Actualmente dos lenguajes de Descripción de Hardware son los mas utilizados.

- **Verilog o su evolución SystemVerilog:**

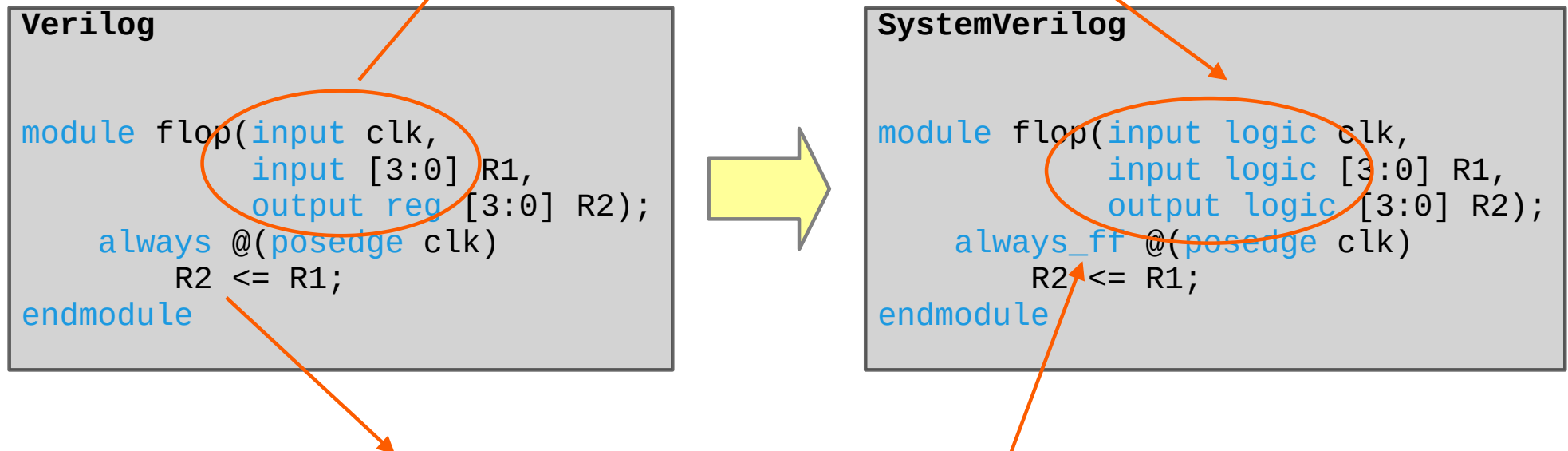
- Desarrollado por Gateway Design Automation en 1985.
- Estándar Abierto desde 1990.
- Posee una sintaxis similar al C.

- **VHDL:**

- Desarrollado por el departamento de defensa de EEUU inicios del 80.
- VHSIC (Very High Speed Integrated Circuit) y HDL (Hardware Description Language).
- IEEE lo estandarizó en 1987.
- Sus raíces son del lenguaje ADA.

Verilog y SystemVerilog

- En los siguientes capítulos los ejemplos se describen en lenguaje de SystemVerilog
- SystemVerilog difiere en algunos detalles con Verilog,
- Por ejemplo elimina `reg` y `wire`, utiliza en su lugar `logic`, excepto aquellos puertos que acepten otros estados



Otra modificación y para evitar problema en el sintetizado, se agregan `always_ff`, `always_comb` y `always_latch`



Módulo

Cuando tenemos diseños complejos, es conveniente dividir el problema, aquí aparecen los módulos.

- Bloque de hardware delimitado que cuenta con un conjunto de entradas (inputs) y salidas (outputs) bien definidas
Ejemplos: una compuerta AND, un multiplexor
- Permiten aplicar los principios de jerarquía y modularidad, dividiendo diseños complejos en bloques más pequeños y reutilizables

Estilos de Descripción de Módulos

Existen dos formas principales de especificar la función de un módulo en HDL:

- **Modelado Comportamental** (Behavioral Modeling):
Describe qué hace el módulo mediante ecuaciones lógicas o expresiones que especifican la relación directa entre las entradas y las salidas.

SystemVerilog

```
module sillyfunction(input logic a, b, c, output logic y);  
    assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b & c;  
endmodule
```

$$y = (\bar{a} \cdot \bar{b} \cdot \bar{c}) + (a \cdot \bar{b} \cdot \bar{c}) + (a \cdot \bar{b} \cdot c)$$

Estilos de Descripción de Módulos

Existen dos formas principales de especificar la función de un módulo en HDL:

- **Modelado Comportamental**
- **Modelado Estructural** (Structural Modeling): Describe cómo está construido el módulo a partir de la interconexión e instanciación de módulos o componentes más simples.

SystemVerilog

```
module sillyfunction(input logic a, b, c, output logic y);  
    logic not_a, not_b, not_c;  
    logic m1, m2, m3;  
    not g_not1 (not_a, a);  
    not g_not2 (not_b, b);  
    not g_not3 (not_c, c);  
    and g_and1 (m1, not_a, not_b, not_c); // ~a & ~b & ~c  
    and g_and2 (m2, a, not_b, not_c); // a & ~b & ~c  
    and g_and3 (m3, a, not_b, c); // a & ~b & c  
    or g_or1 (y, m1, m2, m3);  
endmodule
```

$$y = (\bar{a} \cdot \bar{b} \cdot \bar{c}) + (a \cdot \bar{b} \cdot \bar{c}) + (a \cdot \bar{b} \cdot c)$$



Lógica Combinacional

Los circuitos secuenciales síncronos, están formados por registros y lógica combinatorial

Lógica Combinacional

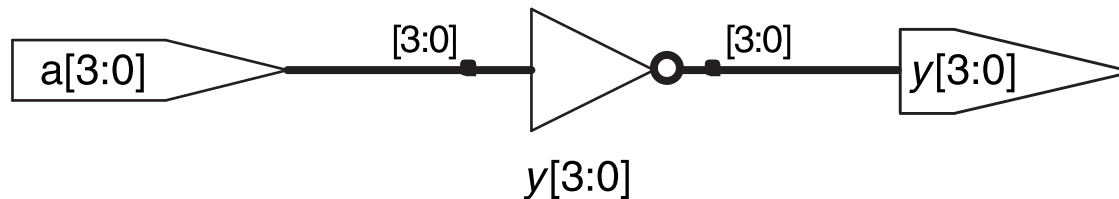
- En estos, las salidas dependen únicamente de las entradas actuales.
- En estos circuitos, nos encontramos con diferentes patrones de diseño;

Lógica Combinacional - Operadores Bit a Bit

- Operan sobre señales de 1 solo bit o en buses de varios bits.
- Para un bus de N bits se define en general como little-endian [N-1:0]

SystemVerilog

```
module inv(input logic [3:0] a,output logic [3:0] y);  
    assign y = ~a;  
endmodule
```



- **assign out = in1 op in2;** es una instrucción de asignación continua (si el lado derecho cambia, el lado izquierdo se recalcula)



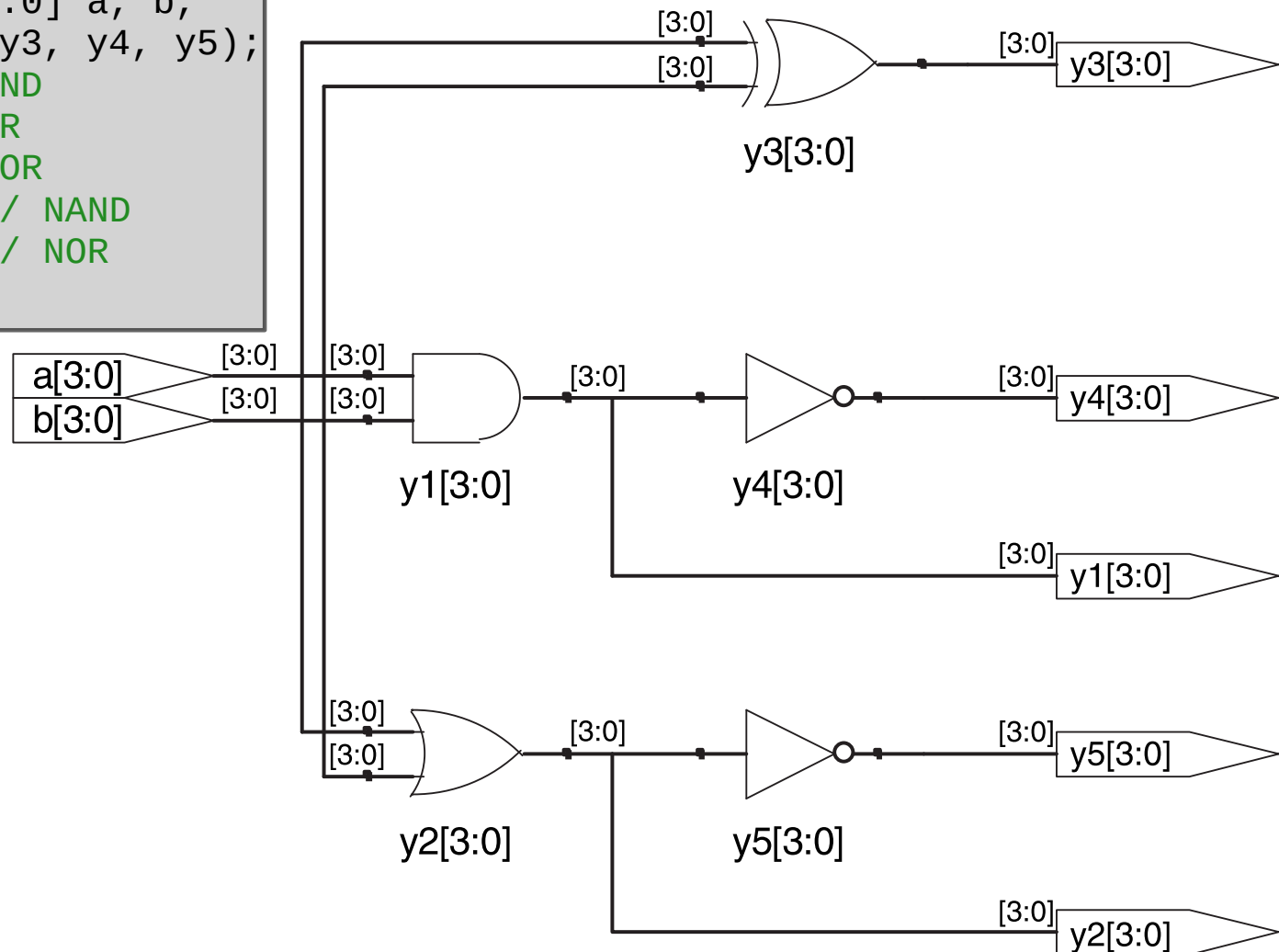
Base de la Lógica Combinacional

Lógica Combinacional - Operadores Bit a Bit

Un ejemplo mas complejo

SystemVerilog

```
module gates(input logic [3:0] a, b,  
output logic [3:0] y1, y2, y3, y4, y5);  
    assign y1 = a & b; // AND  
    assign y2 = a | b; // OR  
    assign y3 = a ^ b; // XOR  
    assign y4 = ~(a & b); // NAND  
    assign y5 = ~(a | b); // NOR  
endmodule
```

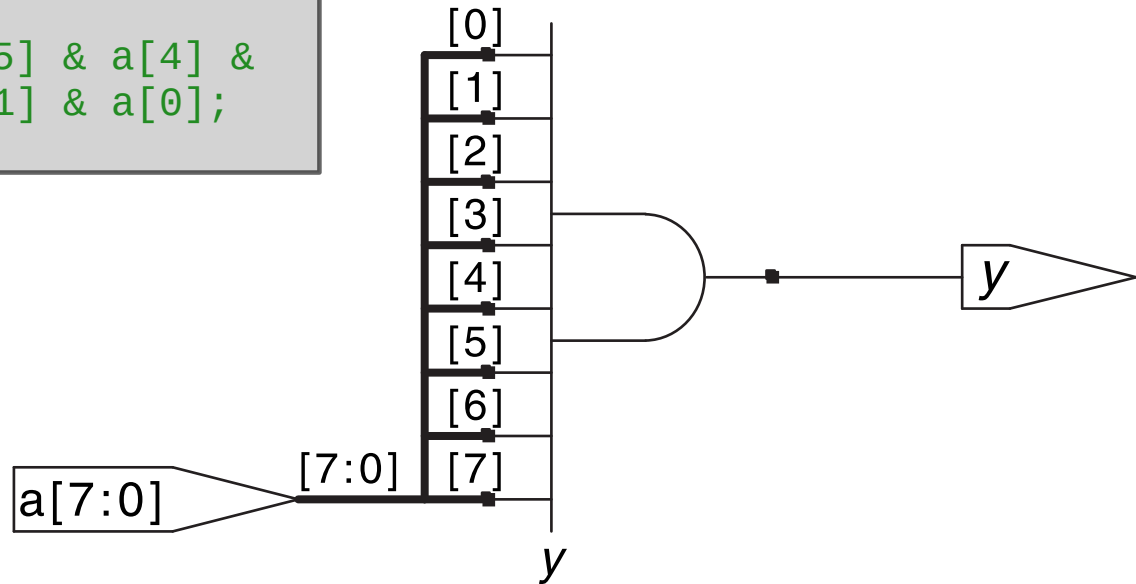


Lógica Combinacional - Operadores de Reducción

- Toman un bus de N bits y entregan una salida de 1 bit al aplicar una operación lógica entre todos los bits del mismo bus.
- Utilizado para determinar paridad o banderas como el cero (Z)

SystemVerilog

```
module and8(input logic [7:0] a, output logic y);  
    assign y = &a;  
    // equivale a y = a[7] & a[6] & a[5] & a[4] &  
    //               a[3] & a[2] & a[1] & a[0];  
endmodule
```



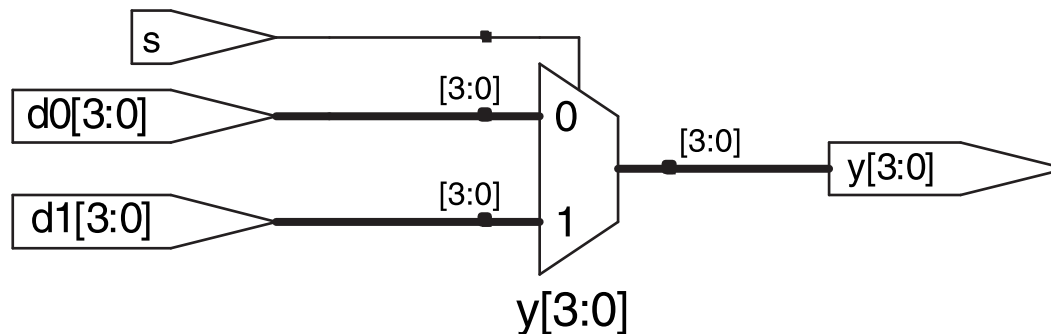
Lógica Combinacional - Asignación Condicional

- Seleccionan la salida entre varias alternativas en función de una condición de entrada.
- El ejemplo muestra un multiplexor

SystemVerilog

```
module mux2(input logic [3:0] d0, d1, input logic s, output logic [3:0] y);  
    assign y = s ? d1 : d0;  
endmodule
```

- El operador ternario **?**, funciona igual que el C, seleccionando la expresión *d1* o *d0* según la expresión *s*

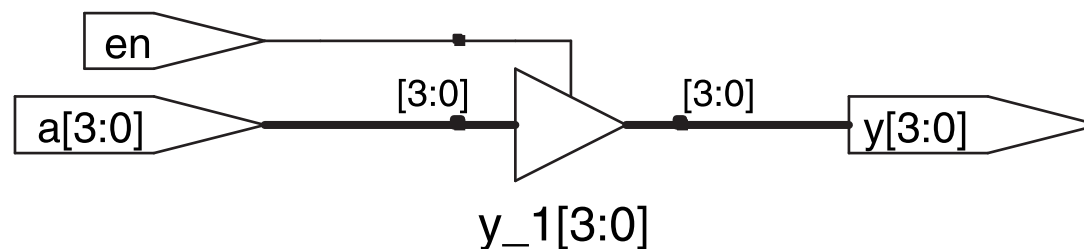


Lógica Combinacional - Búfer de Tristate

- Los HDL definen a la **x** como nivel no válido, en simulación los flip flip se inicializan con **x** y permite detectar olvidos en el reinicio de flip-flip
- La **z** en cambio indica un valor flotante, esto permite crear búfer triestado

SystemVerilog

```
module tristate(input logic [3:0] a, input logic en, output tri [3:0] y);  
    assign y = en ? a : 4'bz;  
endmodule
```





Modelo Estructural

- Describimos ahora el módulo según cómo se construye interconectando submódulos más simples.
- Esto desde la perspectiva de crear sistemas complejos nos permite:
 - **Jerarquía** (Hierarchy): Permite descomponer sistemas complejos en bloques funcionales reutilizables de menor complejidad.
 - **Modularidad y Regularidad**: Fomenta la reutilización de bloques previamente diseñados y probados (como multiplexores, sumadores o decodificadores).

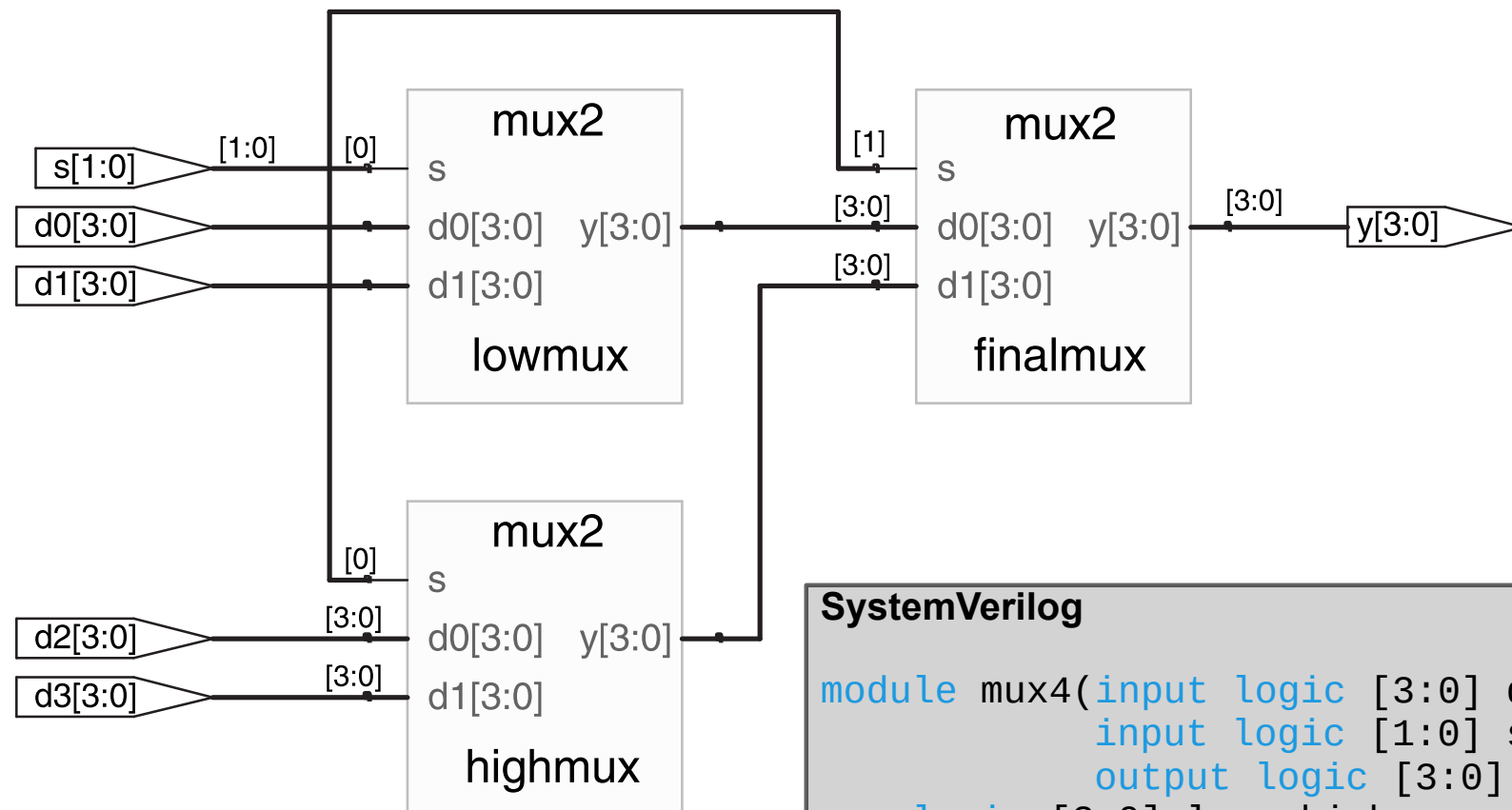


Modelo Estructural

- Cada copia de un submódulo utilizada dentro de una estructura superior se denomina instancia.
- Cada instancia dentro de un módulo superior se indica mediante un nombre distintivo (por ejemplo, *lowmux*, *highmux*, *finalmux*).
- Las entradas y salidas del módulo hijo se interconectan con los cables (wires / logic) o buses del módulo padre.

Modelo Estructural

- Ejemplo de un multiplexor 4:1 a partir de Multiplexores 2:1



SystemVerilog

```
module mux4(input logic [3:0] d0, d1, d2, d3,
            input logic [1:0] s,
            output logic [3:0] y);
    logic [3:0] low, high;
    mux2 lowmux(d0, d1, s[0], low);
    mux2 highmux(d2, d3, s[0], high);
    mux2 finalmux(low, high, s[1], y);
endmodule
```

Modelo Estructural - Ejemplos

- Construir un multiplexor 2:1 a partir de módulos triestado. (ojo a veces no se puede sintetizar)

SystemVerilog

```
module mux2(input logic [3:0] d0, d1, input logic s, output tri [3:0] y);  
    tristate t0(d0, ~s, y);  
    tristate t1(d1, s, y);  
endmodule
```

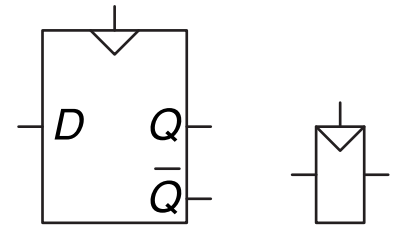
- Multiplexor de 2:1 de 8 bits con dos 2:1 de 4 bits. (cada uno accede a una parte del bus)

SystemVerilog

```
module mux2_8(input logic [7:0] d0, d1, input logic s, output logic [7:0] y);  
    mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);  
    mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);  
endmodule
```

Lógica Secuencial

- Registros formados por Flip-Flops tipo D disparados por flanco ascendente
- En **always** de SystemVerilog, las señales conservan su valor anterior hasta que se produce un evento en la lista de sensibilidad lo que provoca explícitamente su cambio.
- Si en la lista solo usamos la señal de reloj (clk), el sintetizador copia la entrada D en la salida Q únicamente durante la transición del reloj (Memoria).



Base de **Circuitos Secuenciales**

Lógica Secuencial - Registro

SystemVerilog

```
always @(sensitivity list)
    statement;
```

- ***statement*** es ejecutado solo cuando el evento especificado en *sensitivity list* ocurre
- ***always*** se utiliza para flip-flip, latch o lógica combinacional según la lista de sensibilidad y la instrucción.
- Para evitar ambigüedades en el sintetizado se utiliza
 - ***always_ff*** para flip-flip
 - ***always_latch*** para latch
 - ***always_comb*** para combinacionales

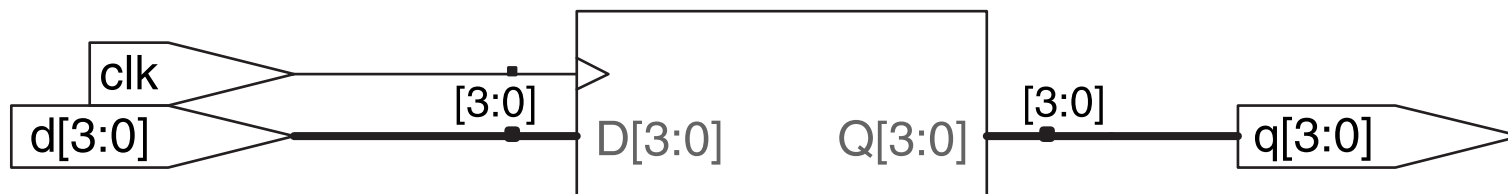
Lógica Secuencial - Registro

- Registro

SystemVerilog

```
module flop(input logic clk, input logic [3:0] d, output logic [3:0] q);  
    always_ff @(posedge clk)  
        q <= d;  
endmodule
```

- $q \leq d$ se ejecuta solo en el flanco ascendente de clk , sino q conserva el valor anterior.
- Esta asignación se denomina no bloqueante y se usa en lugar de `assign` dentro de `always`.



Lógica Secuencial – Registro Reiniciables

Los flip-flop arrancan con un valor x, usar registros reiniciables permite darle un valor de inicio.

- **Asincrónicos:** Responde a un flanco activo de la señal de reset

SystemVerilog

```
module flopr(input logic clk, input logic reset, input logic [3:0] d,
            output logic [3:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule
```

- **Sincrónicos:** Solo evalúa la condición de reset en el siguiente flanco activo del reloj

SystemVerilog

```
module flopr(input logic clk, input logic reset, input logic [3:0] d,
            output logic [3:0] q);
    always_ff @(posedge clk)
        if (reset) q <= 4'b0;
        else q <= d;
endmodule
```

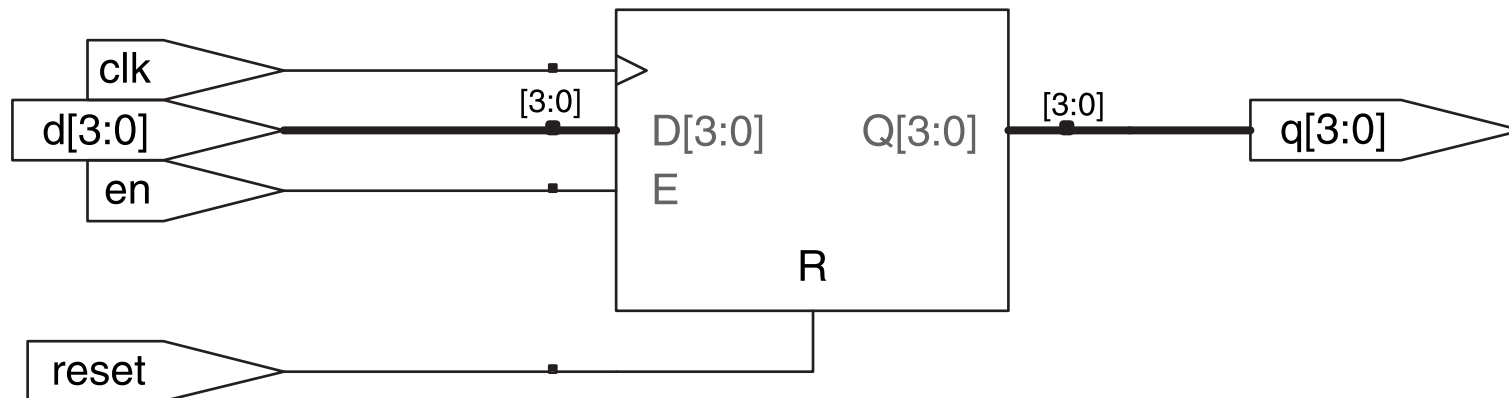
Lógica Secuencial – Registros con Habilitación

Únicamente actualizan su contenido cuando la señal de habilitación está activa.

SystemVerilog

```
module flopenr(input logic clk, input logic reset, input logic en, input logic [3:0] d,
               output logic [3:0] q);
    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 4'b0;
        else if (en) q <= d;
endmodule
```

- Fundamental en el diseño de arquitecturas de procesadores para congelar/habilitar registros específicos.



Lógica Combinacional usando `always_comb`

- Para asignaciones sencillas → ***assign***
- Combinacionales mas complejos requieren otra herramienta
- Uso de ***always_comb***: al igual que *always @(*)* de Verilog
Cada vez que cualquier señal del lado derecho del = cambie, se revalúan todas las instrucciones.
- Ejemplo: inversor de 4 bits

SystemVerilog

```
module inv(input logic [3:0] a, output logic [3:0] y);  
    always_comb  
        y = ~a;  
endmodule
```




Lógica Combinacional usando `always_comb`

`always_comb` tiene la ventaja de:

- Detectar si es combinacional: hay valor de salida para toda las combinaciones.
 - Ejemplo: un if que no tenga else o la falta de default en un case → la salida mantiene el valor anterior → secuencial.
- Permite usar estructuras complejas (case, if else, etc).
- Asignación bloqueante, esto permite variables intermedias reutilizables y control del flujo.

Lógica Combinacional usando `always_comb`

Estructuras en `always_comb`

- Sumador completo

SystemVerilog

```
module fulladder(input logic a, b, cin,output logic s, cout);  
    logic p, g;  
    always_comb  
    begin  
        p = a ^ b;  
        g = a & b;  
        s = p ^ cin;  
        cout = g | (p & cin);  
    end  
endmodule
```

- En estos casos se utiliza el '=' (bloqueante) en lugar de '<=' (no bloqueante) utilizado en los circuitos secuenciales
- Cuando se utiliza mas de una linea dentro del `always_comb`, se debe agregar un *begin / end*

Lógica Combinacional usando `always_comb`

Estructuras en `always_comb`

- Ejemplo de un 7 segmentos usando la clausula `case`

SystemVerilog

```
module sevenseg(input logic [3:0] data,output logic [6:0] segments);
    always_comb
        case(data)
            // abc_defg
            0:      segments = 7'b111_1110;
            1:      segments = 7'b011_0000;
            2:      segments = 7'b110_1101;
            3:      segments = 7'b111_1001;
            4:      segments = 7'b011_0011;
            5:      segments = 7'b101_1011;
            6:      segments = 7'b101_1111;
            7:      segments = 7'b111_0000;
            8:      segments = 7'b111_1111;
            9:      segments = 7'b111_0011;
            default: segments = 7'b000_0000;
        endcase
    endmodule
```

Cubre todas las combinaciones

Lógica Combinacional usando `always_comb`

Estructuras en `always_comb`

- Ejemplo de un decodificador usando `case`

SystemVerilog

```
module decoder3_8(input logic [2:0] a,output logic [7:0] y);  
    always_comb  
        case(a)  
            3'b000: y = 8'b00000001;  
            3'b001: y = 8'b00000010;  
            3'b010: y = 8'b00000100;  
            3'b011: y = 8'b00001000;  
            3'b100: y = 8'b00010000;  
            3'b101: y = 8'b00100000;  
            3'b110: y = 8'b01000000;  
            3'b111: y = 8'b10000000;  
            default: y = 8'bxxxxxxxx;  
        endcase  
endmodule
```

Todas las combinaciones fueron definidas, solo útil en simulación si `a` tiene `z` o `x`

Lógica Combinacional usando `always_comb`

Estructuras en `always_comb`

- Ejemplo de un circuito de prioridad

SystemVerilog

```
module priorityckt(input logic [3:0] a,output logic [3:0] y);  
    always_comb  
        if (a[3])      y = 4'b1000;  
        else if (a[2]) y = 4'b0100;  
        else if (a[1]) y = 4'b0010;  
        else if (a[0]) y = 4'b0001;  
        else           y = 4'b0000;  
endmodule
```

Ojo, para ser combinacional debe
contener todas las combinaciones
posibles

Lógica Combinacional usando `always_comb`

Estructuras en `always_comb`

- Ejemplo de un circuito de prioridad con `casez`

SystemVerilog

```
module priority_casez(input logic [3:0] a, output logic [3:0] y);  
    always_comb  
        casez(a)  
            4'b1???: y = 4'b1000;  
            4'b01??: y = 4'b0100;  
            4'b001?: y = 4'b0010;  
            4'b0001: y = 4'b0001;  
            default: y = 4'b0000;  
        endcase  
    endmodule
```

El `casez` funciona igual que el `case` pero permite el **?** (no me importa)

Ejemplo

- Construir un multiplexor 8:1 de 4 bits llamado mux8 con entradas $S_{2:0}$, d0, d1, d2, d3, d4, d5, d6, d7, y salida y

SystemVerilog

```
module mux8 (input logic [4:0] d0, d1, d2, d3, d4, d5, d6, d7 input logic [2:0] s,
             output logic [4:0] y);
    always_comb
        case (s)
            0: y = d0;
            1: y = d1;
            2: y = d2;
            3: y = d3;
            4: y = d4;
            5: y = d5;
            6: y = d6;
            7: y = d7;
        endcase
endmodule
```

Parametrizado

HDL permiten anchos de bits variables mediante módulos parametrizados.

Multiplexor parametrizado 2 a 1 de 8 bits de ancho por defecto

SystemVerilog

```
module mux2
    #(parameter width = 8)
    (input logic [width-1:0] d0, d1, input logic s,
     output logic [width-1:0] y);
    assign y = s ? d1 : d0;
endmodule
```

Multiplexor 4 a 1 usando mux2 con el ancho por defecto

SystemVerilog

```
module mux4_8(input logic [11:0] d0, d1, d2, d3,
              input logic [1:0] s, output logic [11:0] y);
    logic [11:0] low, hi;
    mux2 lowmux(d0, d1, s[0], low);
    mux2 himux(d2, d3, s[0], hi);
    mux2 outmux(low, hi, s[1], y);
endmodule
```


Parametrizado

HDL permiten anchos de bits variables mediante módulos parametrizados.

Multiplexor parametrizado 2 a 1 de 8 bits de ancho por defecto

SystemVerilog

```
module mux2
    #(parameter width = 8)
    (input logic [width-1:0] d0, d1, input logic s,
     output logic [width-1:0] y);
    assign y = s ? d1 : d0;
endmodule
```

Multiplexor 4 a 1 usando mux2 con 12 bit de ancho

SystemVerilog

```
module mux4_12(input logic [11:0] d0, d1, d2, d3,
               input logic [1:0] s, output logic [11:0] y);
    logic [11:0] low, hi;
    mux2 #(12) lowmux(d0, d1, s[0], low);
    mux2 #(12) himux(d2, d3, s[0], hi);
    mux2 #(.width(12)) outmux(low, hi, s[1], y);
endmodule
```

Ejemplo

- Parametrizar el multiplexor 8:1

SystemVerilog

```
module mux8
    #(parameter width = 4)
    (input logic [width-1:0] d0, d1, d2, d3, d4, d5, d6, d7 input logic [2:0] s,
     output logic [width-1:0] y);
    always_comb
        case (s)
            0: y = d0;
            1: y = d1;
            2: y = d2;
            3: y = d3;
            4: y = d4;
            5: y = d5;
            6: y = d6;
            7: y = d7;
        endcase
endmodule
```

Ejemplo

- Escriba un módulo estructural para calcular la función lógica

$y = a\bar{b} + \bar{b}\bar{c} + \bar{a}bc$, utilizando lógica de multiplexor

#	a	b	c	y
0	0	0	0	
1	0	0	1	
2	0	1	0	
3	0	1	1	
4	1	0	0	
5	1	0	1	
6	1	1	0	
7	1	1	1	

SystemVerilog

```
module ex4_9 (input logic a, b, c, output logic y);  
    mux8 #(1) mux8_1(1'b1, 1'b0, 1'b0, 1'b1, 1'b1, 1'b1, 1'b0, 1'b0, {a,b,c}, y);  
endmodule
```

Bloqueante y no Bloqueante

Tenemos en 1er lugar asignaciones **bloqueantes**.

- Ejemplo: a , b , cin son cero, entonces a cambia a 1 y activa el always, las asignaciones se evalúan en orden

SystemVerilog

```
module fulladder(input logic a, b, cin,
                 output logic s, cout);
    logic p, g;
    always_comb
    begin
        p = a ^ b;           // 1)    p ← 1 ⊕ 0 = 1
        g = a & b;           // 2)    g ← 1 · 0 = 0
        s = p ^ cin;        // 3)    s ← 1 ⊕ 0 = 1
        cout = g | (p & cin); // 4)    cout ← 0 + 1 · 0 = 0
    end
endmodule
```

p y g ya tienen su valor a evaluarse
s y cout

Lo que necesitamos

Bloqueante y no Bloqueante

Tenemos en ahora asignaciones **no bloqueantes**.

- Ejemplo: a , b , cin son cero, entonces a cambia a 1, activa el `always`, las asignaciones ahora se evalúan de manera simultánea

SystemVerilog

```
module fulladder(input logic a, b, cin,  
                 output logic s, cout);
```

```
    logic p, g;
```

```
    always_comb
```

```
    begin
```

```
        p <= a ^ b;
```

```
        g <= a & b;
```

```
        s <= p ^ cin;
```

```
        cout <= g | (p & cin);
```

```
    end
```

```
endmodule
```

No recomendable

// 1era evaluación

$p \leftarrow 1 \oplus 0 = 1$

$g \leftarrow 1 \cdot 0 = 0$

$s \leftarrow 0 \oplus 0 = 0$

$cout \leftarrow 0 + 0 \cdot 0 = 0$

// 2da evaluación

$p \leftarrow 1 \oplus 0 = 1$

$g \leftarrow 1 \cdot 0 = 0$

$s \leftarrow 1 \oplus 0 = 1$

$cout \leftarrow 0 + 1 \cdot 0 = 0$

- s ahora no cambia a 1, pero como p si lo hace, se activa de nuevo al `always` y dispara una segunda evaluación
- La 2da evaluación devuelve $s=1$ pero requirió 2 simulaciones mientras que el hardware es el mismo

Bloqueante y no Bloqueante

Ahora en sistemas secuenciales con asignaciones **no bloqueantes**.

- Ejemplo: $d=0$, $n1=1$ y $q = 0$, en el flanco ascendente tenemos dos asignaciones simultáneas

SystemVerilog

```
module sync(input logic clk, input logic d, output logic q);  
    logic n1;  
    always_ff @(posedge clk)  
        begin  
            n1 <= d;           // n1 ← d = 0  
            q <= n1;          // q ← n1 = 1  
        end  
endmodule
```

Lo que necesitamos

Bloqueante y no Bloqueante

Si ahora lo hacemos con asignaciones **bloqueantes**.

- Ejemplo: $d=0$, $n1=1$ y $q = 0$, en el flanco ascendente tenemos dos asignaciones pero en secuencia, entonces

SystemVerilog

```
module sync(input logic clk, input logic d, output logic q);  
    logic n1;  
    always_ff @(posedge clk)  
        begin  
            n1 = d;           // 1) n1 ← d = 0  
            q = n1;           // 2) q ← n1 = 0  
        end  
endmodule
```

No recomendable

- Paso 1 d se copia a $n1$, quedando $n1$ e 0
- Paso 2 $n1$ se copiar a q , pero $n1$ es ahora 0, q resulta también en 0



Bloqueante y no Bloqueante

Conclusiones

- **Asignaciones Bloqueantes (=)**

- Se evalúan en el orden secuencial.
La variable toma el valor inmediatamente y puede ser usada por las líneas siguientes dentro del mismo bloque.
- Uso obligatorio: Lógica combinacional.

- **Asignaciones No Bloqueantes (<=)**

- Se evalúan concurrentemente.
El estado de todas las entradas del lado derecho se "congela" al iniciar el bloque y las salidas del lado izquierdo se actualizan al final del paso de tiempo.
- Uso obligatorio: Elementos Secuenciales.



¿ Preguntas ?